

FACULDADE INSTED
ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

JOÃO VICTOR MULLER MIRANDA
JULIANO DOS SANTOS APOLINARIO ARAUJO
JÚLIO CÉSAR ZADI DE ASSIS DOS SANTOS
LUAN FLORES MARTINS

**BANCO DE DADOS DE UMA CONCESSIONÁRIA DE VEÍCULOS: modelagem e
implementação em PostgreSQL**

CAMPO GRANDE
2026

JOÃO VICTOR MULLER MIRANDA
JULIANO DOS SANTOS APOLINARIO ARAUJO
JÚLIO CÉSAR ZADI DE ASSIS DOS SANTOS
LUAN FLORES MARTINS

**BANCO DE DADOS DE UMA CONCESSIONÁRIA DE VEÍCULOS: modelagem e
implementação em PostgreSQL**

Trabalho apresentado à disciplina de
Laboratório de Banco de Dados do curso
de Análise e Desenvolvimento de Sistemas
da Faculdade Insted, como requisito
parcial para avaliação.

Professor: Odirley Franco

RESUMO

Este trabalho apresenta a implementação básica de um banco de dados relacional para um sistema de gestão de uma concessionária de veículos, que controla o cadastro de fornecedores, veículos, clientes e funcionários e o registro das vendas. Foi escolhido o Sistema Gerenciador de Banco de Dados (SGBD) PostgreSQL 18, por ser gratuito, de código aberto, aderente ao padrão SQL e oferecer transações, chaves estrangeiras e restrições de verificação. A modelagem foi desenvolvida em dois níveis: o Modelo Entidade-Relacionamento (MER), conceitual, e o Diagrama Entidade-Relacionamento (DER), lógico, complementados por um dicionário de dados. Em seguida, foram criados o banco de dados e a estrutura de cinco tabelas com comandos da linguagem de definição de dados, e realizada a carga de dados com os comandos INSERT, UPDATE e DELETE. Todos os comandos foram executados no PostgreSQL 16.13, e testes de integridade confirmaram que o SGBD recusa operações que deixariam os dados inconsistentes, como vender duas vezes o mesmo veículo ou excluir um cliente que possui compras registradas.

Palavras-chave: Banco de dados. PostgreSQL. Modelagem de dados. SQL. Concessionária de veículos.

LISTA DE FIGURAS

Figura 1 – Modelo Entidade-Relacionamento (MER) do sistema.....	14
Figura 2 – Diagrama Entidade-Relacionamento (DER) do banco concessionaria_db.....	16

LISTA DE QUADROS

Quadro 1 – Comparação entre os SGBDs considerados.....	10
Quadro 2 – Regras de negócio do sistema.....	12
Quadro 3 – Relacionamentos do MER e suas cardinalidades.....	15
Quadro 4 – Chaves primárias, chaves estrangeiras e ações referenciais.....	17
Quadro 5 – Dicionário de dados da tabela fornecedores.....	18
Quadro 6 – Dicionário de dados da tabela veiculos.....	18
Quadro 7 – Dicionário de dados da tabela clientes.....	19
Quadro 8 – Dicionário de dados da tabela funcionarios.....	19
Quadro 9 – Dicionário de dados da tabela vendas.....	19
Quadro 10 – Restrições criadas no banco concessionaria_db.....	24
Quadro 11 – Tabela veiculos após a inserção.....	26
Quadro 12 – Tabela vendas após o registro das vendas.....	27
Quadro 13 – Valores antes e depois das atualizações.....	28
Quadro 14 – Resultado dos testes de integridade.....	30
Quadro 15 – Quantidade de registros por tabela ao final da carga.....	31
Quadro 16 – Veículos em estoque, com o fornecedor.....	31
Quadro 17 – Vendas realizadas.....	32
Quadro 18 – Total vendido por funcionário.....	32
Quadro 19 – Comandos SQL utilizados na implementação.....	32

LISTA DE CÓDIGOS

Código 1 – Criação do banco de dados (arquivo 01_criar_banco.sql).....	21
Código 2 – Criação da tabela fornecedores.....	22
Código 3 – Criação da tabela veiculos.....	23
Código 4 – Criação da tabela clientes.....	23
Código 5 – Criação da tabela funcionarios.....	23
Código 6 – Criação da tabela vendas.....	24
Código 7 – Consulta às restrições criadas no banco.....	24
Código 8 – Inserção dos cadastros iniciais.....	25
Código 9 – Registro das vendas em transações.....	27
Código 10 – Atualizações de dados.....	28
Código 11 – Exclusões de dados.....	29
Código 12 – Comandos que violam as regras de negócio.....	29
Código 13 – Consultas de verificação.....	30

LISTA DE ABREVIATURAS E SIGLAS

ABNT	Associação Brasileira de Normas Técnicas
ACID	Atomicidade, Consistência, Isolamento e Durabilidade
CNPJ	Cadastro Nacional da Pessoa Jurídica
CPF	Cadastro de Pessoas Físicas
DDL	Data Definition Language (Linguagem de Definição de Dados)
DER	Diagrama Entidade-Relacionamento
DML	Data Manipulation Language (Linguagem de Manipulação de Dados)
DQL	Data Query Language (Linguagem de Consulta de Dados)
FK	Foreign Key (chave estrangeira)
MER	Modelo Entidade-Relacionamento
PK	Primary Key (chave primária)
SGBD	Sistema Gerenciador de Banco de Dados
SQL	Structured Query Language (Linguagem de Consulta Estruturada)
TCL	Transaction Control Language (Linguagem de Controle de Transações)
UK	Unique Key (chave única)

SUMÁRIO

1	INTRODUÇÃO.....	8
2	OBJETIVOS.....	9
2.1	OBJETIVO GERAL.....	9
2.2	OBJETIVOS ESPECÍFICOS.....	9
3	ESCOLHA DO BANCO DE DADOS.....	10
4	MODELAGEM DE DADOS.....	12
4.1	DESCRIÇÃO DO SISTEMA E REGRAS DE NEGÓCIO.....	12
4.2	MODELO ENTIDADE-RELACIONAMENTO (MER).....	12
4.3	DIAGRAMA ENTIDADE-RELACIONAMENTO (DER).....	15
4.4	DICIONÁRIO DE DADOS.....	18
5	IMPLEMENTAÇÃO.....	21
5.1	CRIAÇÃO DO BANCO DE DADOS.....	21
5.2	CRIAÇÃO DA ESTRUTURA DE TABELAS.....	22
5.3	CARGA DE DADOS.....	25
5.3.1	Inserção de dados (INSERT).....	25
5.3.2	Registro das vendas (INSERT e UPDATE em transação).....	27
5.3.3	Atualização de dados (UPDATE).....	28
5.3.4	Exclusão de dados (DELETE).....	29
5.4	TESTES DE INTEGRIDADE.....	29
5.5	CONSULTAS DE VERIFICAÇÃO.....	30
5.6	RESUMO DOS COMANDOS UTILIZADOS.....	32
6	CONCLUSÃO.....	33
	REFERÊNCIAS.....	34
	APÊNDICE A – SCRIPT DE CRIAÇÃO DO BANCO.....	35
	APÊNDICE B – SCRIPT DE ESTRUTURA E CARGA DE DADOS.....	36

1 INTRODUÇÃO

Uma concessionária de veículos lida diariamente com um grande volume de informações: os veículos que chegam dos fornecedores, os que estão em estoque, reservados ou já vendidos, os dados dos clientes, a equipe de vendas e o histórico de cada negociação. Quando essas informações ficam espalhadas em planilhas ou anotações, surgem problemas conhecidos, como o mesmo veículo vendido duas vezes, cadastros de clientes duplicados e a dificuldade de saber quanto cada vendedor negociou no mês.

Um banco de dados relacional resolve esses problemas ao reunir as informações em um único lugar, organizadas em tabelas relacionadas entre si, e ao entregar a um Sistema Gerenciador de Banco de Dados (SGBD) a tarefa de fazer cumprir as regras do negócio. Segundo Elmasri e Navathe (2018), o SGBD é um conjunto de programas que permite aos usuários criar e manter um banco de dados, facilitando a definição, a construção, a manipulação e o compartilhamento dos dados entre diversos usuários e aplicações.

Este trabalho apresenta a implementação básica do banco de dados de um sistema para concessionária de veículos, produto escolhido pelo grupo para desenvolvimento. O foco está na camada de dados: não faz parte do escopo o desenvolvimento de telas (*front-end*), e sim a definição do SGBD, a modelagem dos dados, a criação do banco e de suas tabelas e a carga e manipulação dos dados por meio da linguagem SQL.

O documento está organizado da seguinte forma: a seção 2 apresenta os objetivos; a seção 3 justifica a escolha do SGBD; a seção 4 descreve a modelagem de dados, com o MER, o DER e o dicionário de dados; a seção 5 detalha a implementação, com todos os comandos utilizados e os resultados obtidos; e a seção 6 traz as conclusões. O script SQL completo está no Apêndice A. A formatação segue a NBR 14724 (ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS, 2011) e as referências, a NBR 6023 (ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS, 2018).

2 OBJETIVOS

2.1 OBJETIVO GERAL

Implementar um banco de dados relacional para um sistema de gestão de concessionária de veículos, contemplando a definição do SGBD, a criação do banco de dados, a criação da estrutura de tabelas e a carga de dados com os comandos INSERT, UPDATE e DELETE.

2.2 OBJETIVOS ESPECÍFICOS

- a) definir e justificar o SGBD utilizado;
- b) levantar as regras de negócio e as entidades do sistema;
- c) elaborar o Modelo Entidade-Relacionamento (MER) e o Diagrama Entidade-Relacionamento (DER);
- d) elaborar o dicionário de dados de todas as tabelas;
- e) criar o banco de dados e as tabelas, com chaves primárias, chaves estrangeiras e restrições de integridade;
- f) inserir, atualizar e excluir dados, registrando todos os comandos utilizados;
- g) verificar, por meio de consultas e testes, que os dados permanecem íntegros após as operações.

3 ESCOLHA DO BANCO DE DADOS

O SGBD escolhido foi o **PostgreSQL**, na versão 18. O PostgreSQL é um SGBD objeto-relacional de código aberto, desenvolvido por uma comunidade internacional, o PostgreSQL Global Development Group, e distribuído gratuitamente sob a licença PostgreSQL, semelhante às licenças BSD e MIT (THE POSTGRES GLOBAL DEVELOPMENT GROUP, 2026). A escolha considerou os seguintes critérios:

- custo e licença: é gratuito e de código aberto, sem edições pagas e sem limites de tamanho de banco, de memória ou de processadores;
- integridade dos dados: oferece transações com as propriedades ACID, chaves estrangeiras com ações referenciais (RESTRICT, CASCADE e SET NULL) e restrições CHECK, e permite até que comandos de definição de dados, como CREATE TABLE, façam parte de uma transação;
- aderência ao padrão SQL: segue de perto o padrão da linguagem, inclusive nas colunas de identidade (GENERATED ALWAYS AS IDENTITY) usadas neste trabalho para gerar as chaves primárias;
- adoção no mercado: é amplamente usado em empresas e oferecido pelos principais provedores de nuvem, com documentação oficial completa e comunidade ativa;
- ferramentas: o pgAdmin, gratuito, permite criar o banco, executar os scripts e visualizar os resultados em ambiente gráfico, e o psql permite fazer o mesmo pelo terminal.

O Quadro 1 compara o PostgreSQL com outros dois SGBDs considerados pelo grupo.

Quadro 1 – Comparação entre os SGBDs considerados

Critério	PostgreSQL 18	MySQL 8.0	SQL Server Express
Licença	Código aberto (PostgreSQL License)	Código aberto (GPL) e edições comerciais	Proprietária
Custo	Gratuito	Gratuito na edição Community	Gratuito, com limites de recursos
Transações e chaves estrangeiras	Sim	Sim (motor InnoDB)	Sim
Restrições CHECK	Sim	Sim (desde a 8.0.16)	Sim
CREATE TABLE dentro de transação	Sim	Não (confirma a transação automaticamente)	Sim
Ferramenta gráfica mais usada	pgAdmin	MySQL Workbench	SQL Server Management Studio
Sistemas operacionais	Windows, Linux e macOS	Windows, Linux e macOS	Windows e Linux

Fonte: Elaborado pelos autores com base na documentação oficial de cada SGBD (2026).

Os três atenderiam ao escopo do trabalho. O PostgreSQL foi escolhido por ser totalmente gratuito e aberto, sem edição comercial nem limites de uso, por seguir de perto o padrão SQL e por oferecer todos os recursos de integridade exigidos pelas regras de negócio da concessionária. Os comandos deste trabalho foram executados e validados no PostgreSQL 16.13, com a codificação UTF8, que representa corretamente os acentos da língua portuguesa. Os scripts usam apenas recursos disponíveis desde a versão 10 do PostgreSQL, como as colunas de identidade, e por isso funcionam da mesma forma na versão 18, instalada nos computadores do grupo.

4 MODELAGEM DE DADOS

A modelagem de dados foi feita em etapas, como recomenda Heuser (2009): primeiro o levantamento das regras de negócio; depois o modelo conceitual, independente de SGBD, que descreve os dados do ponto de vista do negócio; e por fim o modelo lógico, que já define as tabelas, colunas e chaves a serem implementadas.

4.1 DESCRIÇÃO DO SISTEMA E REGRAS DE NEGÓCIO

O sistema controla o funcionamento básico de uma concessionária: o cadastro de fornecedores, veículos, clientes e funcionários e o registro das vendas, que relacionam o cliente comprador, o funcionário responsável e o veículo vendido. As regras de negócio levantadas estão no Quadro 2 e orientaram todas as decisões de modelagem.

Quadro 2 – Regras de negócio do sistema

Código	Regra
RN01	Cada fornecedor é identificado pelo CNPJ, que não pode se repetir.
RN02	Um fornecedor pode fornecer vários veículos, e cada veículo vem de, no máximo, um fornecedor. Se o fornecedor for excluído, o veículo continua no estoque, sem fornecedor associado.
RN03	Cada veículo tem marca, modelo, ano, cor, preço e situação. A placa, quando existir, é única; veículos 0 km ainda não emplacados ficam sem placa.
RN04	A situação do veículo é Disponível, Reservado ou Vendido, e todo veículo cadastrado começa como Disponível.
RN05	Cada cliente é identificado pelo CPF, que não pode se repetir.
RN06	Uma venda envolve exatamente um cliente, um funcionário e um veículo, e registra a data e o valor negociado. Um veículo pode ser vendido no máximo uma vez e, ao ser vendido, passa à situação Vendido.
RN07	Clientes, funcionários e veículos que constam em alguma venda não podem ser excluídos, para preservar o histórico de vendas.
RN08	Preços, salários e valores de venda devem ser maiores que zero.

Fonte: Elaborado pelos autores (2026).

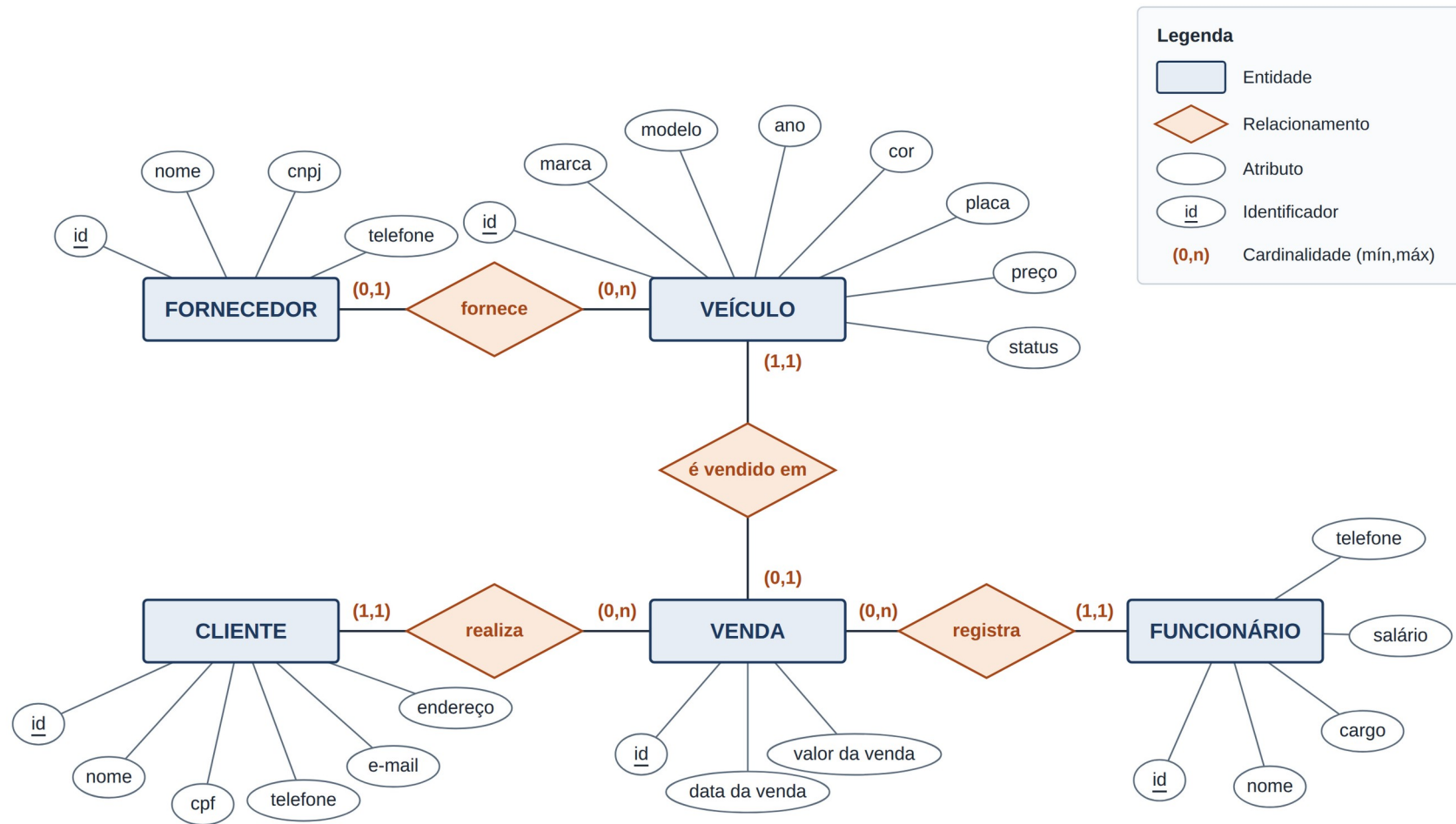
4.2 MODELO ENTIDADE-RELACIONAMENTO (MER)

O Modelo Entidade-Relacionamento, proposto por Chen (1976), descreve os dados de forma conceitual por meio de entidades (objetos do mundo real sobre os quais se deseja guardar informações), atributos (propriedades das entidades) e relacionamentos (associações entre entidades). O MER deste trabalho foi

representado na notação de Chen, com as cardinalidades no formato (mínima, máxima) usado por Heuser (2009): o par escrito junto a uma entidade indica com quantas ocorrências dela uma ocorrência da outra entidade se associa. Os atributos identificadores aparecem sublinhados.

A Figura 1 apresenta o MER do sistema, com cinco entidades e quatro relacionamentos. Por conter muitos atributos, o diagrama foi colocado em uma página na orientação paisagem.

Figura 1 – Modelo Entidade-Relacionamento (MER) do sistema



Fonte: Elaborado pelos autores (2026).

O Quadro 3 explica como ler cada relacionamento do MER.

Quadro 3 – Relacionamentos do MER e suas cardinalidades

Relacioname nto	Entidades	Tipo	Leitura
fornece	FORNECEDOR e VEÍCULO	1:N	Um fornecedor fornece zero ou muitos veículos; um veículo vem de zero ou um fornecedor.
é vendido em	VEÍCULO e VENDA	1:1	Um veículo é vendido em zero ou uma venda; cada venda se refere a exatamente um veículo.
realiza	CLIENTE e VENDA	1:N	Um cliente realiza zero ou muitas compras; cada venda é feita para exatamente um cliente.
registra	FUNCIONÁRIO e VENDA	1:N	Um funcionário registra zero ou muitas vendas; cada venda é registrada por exatamente um funcionário.

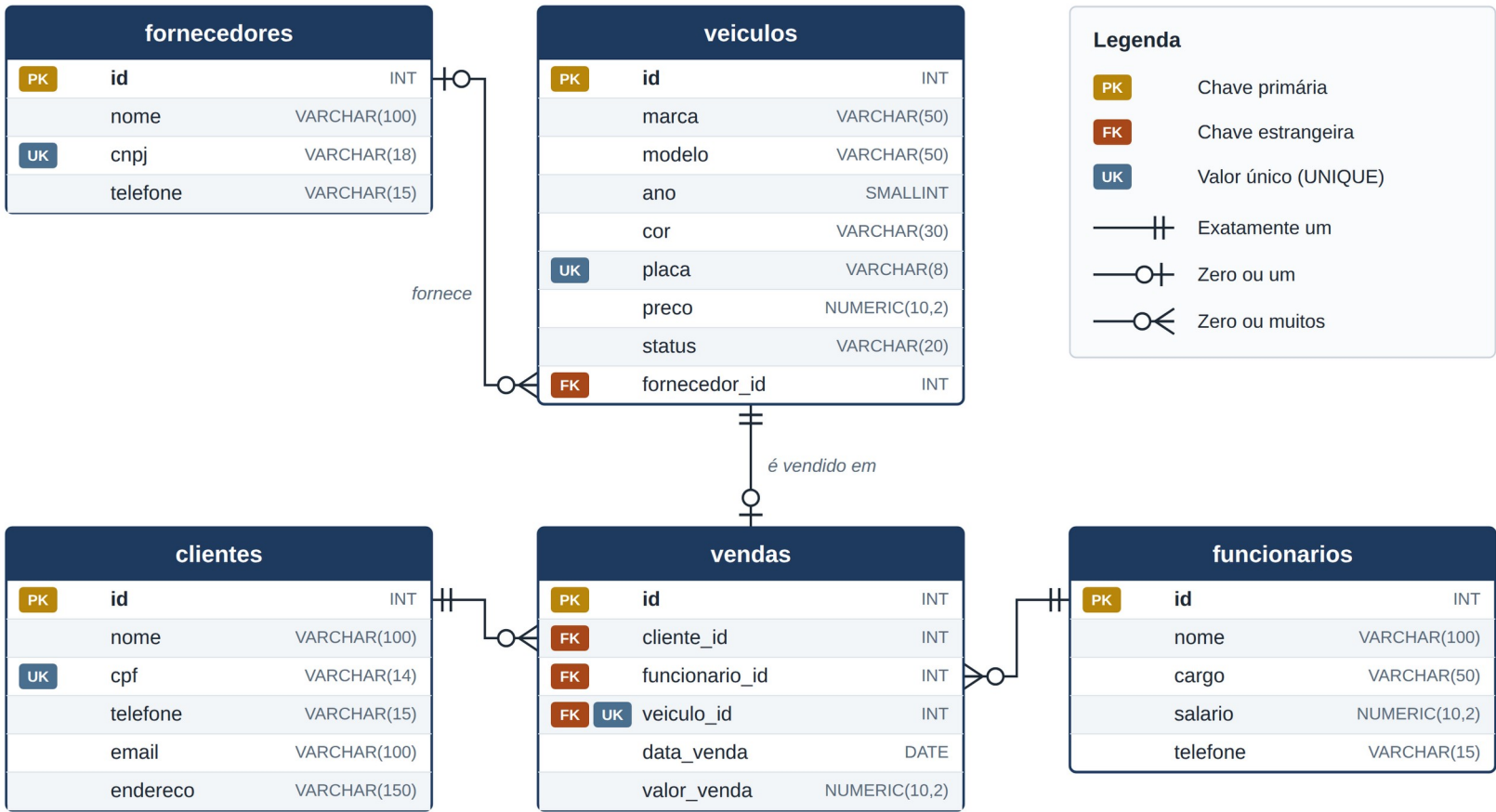
Fonte: Elaborado pelos autores (2026).

A venda foi modelada como entidade, e não como um relacionamento entre cliente, funcionário e veículo, porque possui identidade própria (o número da venda) e atributos próprios (data e valor negociado), além de se associar a três entidades ao mesmo tempo.

4.3 DIAGRAMA ENTIDADE-RELACIONAMENTO (DER)

O DER apresenta o modelo lógico do banco, obtido pela transformação do MER em tabelas (HEUSER, 2009). Foi usada a notação conhecida como pé de galinha (*crow's foot*), a mesma usada pela ferramenta de diagramas do pgAdmin (*ERD Tool*), em que cada tabela aparece com suas colunas, tipos de dados e chaves, e as linhas entre as tabelas indicam as chaves estrangeiras e suas cardinalidades (Figura 2).

Figura 2 – Diagrama Entidade-Relacionamento (DER) do banco concessionaria_db



Fonte: Elaborado pelos autores (2026).

Na transformação do modelo conceitual para o lógico, foram aplicadas as seguintes regras:

- cada entidade virou uma tabela, com nome no plural e sem acentos (fornecedores, veiculos, clientes, funcionarios e vendas), e cada atributo virou uma coluna;
- o identificador de cada entidade virou uma chave primária numérica gerada automaticamente pelo SGBD (GENERATED ALWAYS AS IDENTITY);
- nos relacionamentos 1:N, a chave primária do lado 1 foi copiada como chave estrangeira para a tabela do lado N: fornecedor_id em veiculos, e cliente_id e funcionario_id em vendas;
- no relacionamento 1:1 entre veículo e venda, a chave estrangeira veiculo_id ficou na tabela vendas, com a restrição UNIQUE, que impede que o mesmo veículo apareça em duas vendas;
- os atributos que identificam o registro no mundo real (CNPJ, CPF e placa) receberam a restrição UNIQUE.

O Quadro 4 resume as chaves de cada tabela e as ações referenciais escolhidas para as chaves estrangeiras.

Quadro 4 – Chaves primárias, chaves estrangeiras e ações referenciais

Tabela	Chave primária	Chave estrangeira	Referencia	Ao excluir	Ao atualizar
fornecedores	id	–	–	–	–
veiculos	id	fornecedor_id	fornecedores (id)	SET NULL	CASCADE
clientes	id	–	–	–	–
funcionarios	id	–	–	–	–
vendas	id	cliente_id	clientes (id)	RESTRICT	CASCADE
vendas	id	funcionario_id	funcionarios (id)	RESTRICT	CASCADE
vendas	id	veiculo_id	veiculos (id)	RESTRICT	CASCADE

Fonte: Elaborado pelos autores (2026).

Segundo Date (2004), a regra de integridade referencial determina que o banco não pode conter valores de chave estrangeira sem correspondência na chave primária referenciada. As ações referenciais definem como o SGBD mantém essa regra. A ação RESTRICT nas chaves estrangeiras de vendas garante a regra RN07: o SGBD recusa a exclusão de um cliente, funcionário ou veículo que já participou de uma venda. A ação SET NULL em veiculos implementa a regra RN02: se um fornecedor for excluído, seus veículos permanecem cadastrados, com a coluna fornecedor_id vazia. A ação CASCADE na atualização propaga automaticamente uma eventual mudança de chave primária para as tabelas que a referenciam.

4.4 DICIONÁRIO DE DADOS

O dicionário de dados descreve cada coluna das tabelas: o tipo de dado, se aceita valores nulos, as chaves e restrições a que está sujeita e o seu significado. Os Quadros 5 a 9 apresentam o dicionário das cinco tabelas do banco. Na coluna Chave, PK indica chave primária, FK chave estrangeira e UK valor único; na coluna Nulo, "Não" indica que o preenchimento é obrigatório (NOT NULL).

Quadro 5 – Dicionário de dados da tabela fornecedores

Coluna	Tipo	Nulo	Chave	Padrão / Restrição	Descrição
id	INT	Não	PK	IDENTITY	Código do fornecedor, gerado automaticamente
nome	VARCHAR(100)	Não	–	–	Razão social ou nome fantasia
cnpj	VARCHAR(18)	Não	UK	uq_fornecedor_cnpj	CNPJ no formato 00.000.000/0000-00
telefone	VARCHAR(15)	Sim	–	–	Telefone de contato, com DDD

Fonte: Elaborado pelos autores (2026).

Quadro 6 – Dicionário de dados da tabela veiculos

Coluna	Tipo	Nulo	Chave	Padrão / Restrição	Descrição
id	INT	Não	PK	IDENTITY	Código do veículo, gerado automaticamente
marca	VARCHAR(50)	Não	–	–	Fabricante (ex.: Fiat, Toyota)
modelo	VARCHAR(50)	Não	–	–	Modelo (ex.: Toro, Corolla)
ano	SMALLINT	Não	–	CHECK (ano BETWEEN 1900 AND 2100)	Ano do modelo, com quatro dígitos
cor	VARCHAR(30)	Sim	–	–	Cor predominante
placa	VARCHAR(8)	Sim	UK	uq_veiculo_placa	Placa no padrão Mercosul; nula enquanto o veículo 0 km não é emplacado
preco	NUMERIC(10,2)	Não	–	CHECK (preco > 0)	Preço de venda anunciado, em reais
status	VARCHAR(20)	Não	–	DEFAULT 'Disponivel'; CHECK: Disponivel, Reservado ou Vendido	Situação do veículo no estoque
fornecedor_id	INT	Sim	FK	fornecedores (id), ON DELETE SET NULL	Fornecedor de quem o veículo foi adquirido

Fonte: Elaborado pelos autores (2026).

Quadro 7 – Dicionário de dados da tabela clientes

Coluna	Tipo	Nulo	Chave	Padrão / Restrição	Descrição
id	INT	Não	PK	IDENTITY	Código do cliente, gerado automaticamente
nome	VARCHAR(100)	Não	–	–	Nome completo
cpf	VARCHAR(14)	Não	UK	uq_cliente_cpf	CPF no formato 000.000.000-00
telefone	VARCHAR(15)	Sim	–	–	Telefone, com DDD
email	VARCHAR(100)	Sim	–	–	Endereço de e-mail
endereço	VARCHAR(150)	Sim	–	–	Logradouro, número, cidade e UF

Fonte: Elaborado pelos autores (2026).

Quadro 8 – Dicionário de dados da tabela funcionarios

Coluna	Tipo	Nulo	Chave	Padrão / Restrição	Descrição
id	INT	Não	PK	IDENTITY	Código do funcionário, gerado automaticamente
nome	VARCHAR(100)	Não	–	–	Nome completo
cargo	VARCHAR(50)	Não	–	–	Cargo (ex.: Vendedor, Gerente de Vendas)
salario	NUMERIC(10,2)	Não	–	CHECK (salario > 0)	Salário mensal, em reais
telefone	VARCHAR(15)	Sim	–	–	Telefone, com DDD

Fonte: Elaborado pelos autores (2026).

Quadro 9 – Dicionário de dados da tabela vendas

Coluna	Tipo	Nulo	Chave	Padrão / Restrição	Descrição
id	INT	Não	PK	IDENTITY	Número da venda, gerado automaticamente
cliente_id	INT	Não	FK	clientes (id), ON DELETE RESTRICT	Cliente que comprou o veículo
funcionario_id	INT	Não	FK	funcionarios (id), ON DELETE RESTRICT	Funcionário responsável pela venda
veiculo_id	INT	Não	FK, UK	veiculos (id), ON DELETE RESTRICT; uq_venda_veiculo	Veículo vendido; o UNIQUE impede vendê-lo duas vezes
data_venda	DATE	Não	–	–	Data em que a venda foi fechada
valor_venda	NUMERIC(10,2)	Não	–	CHECK (valor_venda > 0)	Valor negociado, em reais, que pode diferir do preço anunciado

Fonte: Elaborado pelos autores (2026).

Os valores monetários usam o tipo NUMERIC(10,2), numérico exato que evita os erros de arredondamento dos tipos de ponto flutuante e comporta valores de até

99.999.999,99. CPF, CNPJ, telefone e placa foram armazenados como texto (VARCHAR) porque não são usados em cálculos e podem conter zeros à esquerda e caracteres de formatação. Como o PostgreSQL não possui um tipo específico para anos, a coluna ano usa SMALLINT (inteiro de 2 bytes) com uma restrição CHECK que limita o valor a anos plausíveis. Na coluna Padrão / Restrição, IDENTITY indica GENERATED ALWAYS AS IDENTITY: o valor é gerado pelo SGBD e não pode ser informado no INSERT.

5 IMPLEMENTAÇÃO

A implementação seguiu as quatro etapas propostas para o trabalho: definição do SGBD (seção 3), criação do banco de dados, criação da estrutura de tabelas e carga de dados. Os comandos estão em dois arquivos, reproduzidos nos Apêndices A e B: 01_criar_banco.sql, executado conectado ao banco padrão postgres, e 02_tabelas_e_carga.sql, executado conectado ao banco concessionaria_db. Pelo terminal, os dois são executados com `psql -U postgres -f 01_criar_banco.sql` e `psql -U postgres -d concessionaria_db -f 02_tabelas_e_carga.sql`.

A divisão em dois arquivos é necessária porque, no PostgreSQL, cada conexão fica ligada ao banco em que foi aberta: não existe um comando como o USE do MySQL, e para passar a trabalhar no banco recém-criado é preciso abrir uma nova conexão (no pgAdmin, abrindo o Query Tool no banco concessionaria_db; no psql, com o metacomando \c).

Os comandos SQL costumam ser agrupados em subconjuntos: a DDL, que define a estrutura do banco (CREATE, DROP); a DML, que manipula os dados (INSERT, UPDATE, DELETE); a DQL, que consulta os dados (SELECT); e a TCL, que controla as transações (BEGIN, COMMIT). Todos eles aparecem nas etapas a seguir.

5.1 CRIAÇÃO DO BANCO DE DADOS

O Código 1 cria o banco de dados concessionaria_db. O comando DROP DATABASE IF EXISTS apaga uma versão anterior do banco, o que permite executar os scripts várias vezes, sempre a partir do zero. A opção ENCODING = 'UTF8' define a codificação de caracteres do banco, que armazena qualquer caractere Unicode, inclusive os acentos do português. A opção TEMPLATE = template0 cria o banco a partir do modelo original do PostgreSQL, exigido quando a codificação escolhida pode ser diferente da do modelo padrão.

Código 1 – Criação do banco de dados (arquivo 01_criar_banco.sql)

```
-- Apaga uma versão anterior para que o script possa ser executado
-- novamente sem erro
DROP DATABASE IF EXISTS concessionaria_db;

CREATE DATABASE concessionaria_db
    WITH ENCODING = 'UTF8'
    TEMPLATE = template0;
```

Fonte: Elaborado pelos autores (2026).

No pgAdmin, esses dois comandos devem ser executados um de cada vez: o PostgreSQL não permite DROP DATABASE nem CREATE DATABASE dentro de um bloco de transação, e o pgAdmin envia todos os comandos da janela juntos, em um único bloco. Pelo psql, que envia um comando por vez, o arquivo pode ser executado inteiro.

5.2 CRIAÇÃO DA ESTRUTURA DE TABELAS

As tabelas foram criadas com o comando CREATE TABLE, em uma ordem que respeita as dependências entre elas: primeiro fornecedores, depois veículos, que a referencia; em seguida clientes e funcionarios; e por último vendas, que referencia três das anteriores. Uma chave estrangeira só pode ser criada quando a tabela referenciada já existe. Diferentemente do MySQL, o PostgreSQL não exige a escolha de um motor de armazenamento: todas as tabelas têm suporte nativo a chaves estrangeiras e a transações (THE POSTGRESQL GLOBAL DEVELOPMENT GROUP, 2026).

As restrições foram declaradas com nome, pela cláusula CONSTRAINT, o que deixa as mensagens de erro do SGBD mais claras, como se verá na seção 5.4. As restrições utilizadas foram:

- a) PRIMARY KEY com GENERATED ALWAYS AS IDENTITY: identifica cada linha, com numeração gerada pelo SGBD; é a forma do padrão SQL, recomendada no PostgreSQL no lugar do antigo tipo SERIAL;
- b) NOT NULL: torna obrigatório o preenchimento da coluna;
- c) UNIQUE: impede valores repetidos na coluna (CNPJ, CPF, placa e veículo da venda);
- d) DEFAULT: define o valor usado quando a coluna não é informada no INSERT (a situação 'Disponível');
- e) CHECK: valida o conteúdo da coluna (valores maiores que zero, ano plausível e situação dentro da lista permitida);
- f) FOREIGN KEY: liga a tabela a outra e define o que acontece ao excluir ou atualizar a linha referenciada.

Os Códigos 2 a 6 apresentam a criação de cada tabela.

Código 2 – Criação da tabela fornecedores

```
CREATE TABLE fornecedores (  
  id          INT          GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  nome       VARCHAR(100) NOT NULL,  
  cnpj       VARCHAR(18)  NOT NULL,  
  telefone   VARCHAR(15),
```

```

CONSTRAINT uq_fornecedor_cnpj UNIQUE (cnpj)
);

```

Fonte: Elaborado pelos autores (2026).

Código 3 – Criação da tabela veiculos

```

CREATE TABLE veiculos (
    id            INT            GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    marca         VARCHAR(50)    NOT NULL,
    modelo        VARCHAR(50)    NOT NULL,
    ano           SMALLINT       NOT NULL,
    cor           VARCHAR(30),
    placa         VARCHAR(8),
    preco         NUMERIC(10,2) NOT NULL,
    status        VARCHAR(20)    NOT NULL DEFAULT 'Disponivel',
    fornecedor_id INT,
    CONSTRAINT uq_veiculo_placa UNIQUE (placa),
    CONSTRAINT ck_veiculo_ano CHECK (ano BETWEEN 1900 AND 2100),
    CONSTRAINT ck_veiculo_preco CHECK (preco > 0),
    CONSTRAINT ck_veiculo_status
        CHECK (status IN ('Disponivel', 'Reservado', 'Vendido')),
    CONSTRAINT fk_veiculo_fornecedor
        FOREIGN KEY (fornecedor_id) REFERENCES fornecedores (id)
        ON DELETE SET NULL
        ON UPDATE CASCADE
);

```

Fonte: Elaborado pelos autores (2026).

Código 4 – Criação da tabela clientes

```

CREATE TABLE clientes (
    id            INT            GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nome         VARCHAR(100) NOT NULL,
    cpf          VARCHAR(14)    NOT NULL,
    telefone     VARCHAR(15),
    email        VARCHAR(100),
    endereco     VARCHAR(150),
    CONSTRAINT uq_cliente_cpf UNIQUE (cpf)
);

```

Fonte: Elaborado pelos autores (2026).

Código 5 – Criação da tabela funcionarios

```

CREATE TABLE funcionarios (
    id            INT            GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nome         VARCHAR(100) NOT NULL,
    cargo        VARCHAR(50)    NOT NULL,
    salario      NUMERIC(10,2) NOT NULL,
    telefone     VARCHAR(15),
    CONSTRAINT ck_funcionario_salario CHECK (salario > 0)
);

```

Fonte: Elaborado pelos autores (2026).

Código 6 – Criação da tabela vendas

```

CREATE TABLE vendas (
  id          INT          GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  cliente_id  INT          NOT NULL,
  funcionario_id INT      NOT NULL,
  veiculo_id  INT          NOT NULL,
  data_venda  DATE         NOT NULL,
  valor_venda NUMERIC(10,2) NOT NULL,
  CONSTRAINT uq_venda_veiculo UNIQUE (veiculo_id),
  CONSTRAINT ck_venda_valor CHECK (valor_venda > 0),
  CONSTRAINT fk_venda_cliente
    FOREIGN KEY (cliente_id) REFERENCES clientes (id)
    ON DELETE RESTRICT ON UPDATE CASCADE,
  CONSTRAINT fk_venda_funcionario
    FOREIGN KEY (funcionario_id) REFERENCES funcionarios (id)
    ON DELETE RESTRICT ON UPDATE CASCADE,
  CONSTRAINT fk_venda_veiculo
    FOREIGN KEY (veiculo_id) REFERENCES veiculos (id)
    ON DELETE RESTRICT ON UPDATE CASCADE
);

```

Fonte: Elaborado pelos autores (2026).

Para conferir a estrutura criada, o catálogo do próprio SGBD (a tabela de sistema pg_constraint) foi consultado com o Código 7. O resultado, no Quadro 10, mostra as 18 restrições criadas nas cinco tabelas: 5 chaves primárias, 4 restrições UNIQUE, 5 restrições CHECK e 4 chaves estrangeiras. As chaves primárias, que não foram declaradas com nome, receberam do PostgreSQL nomes no formato tabela_pkey. O filtro por contype mantém o resultado igual em todas as versões: a partir do PostgreSQL 18, as restrições NOT NULL também passaram a ser registradas no pg_constraint, com o tipo 'n', e ficariam de fora da contagem acima.

Código 7 – Consulta às restrições criadas no banco

```

SELECT conrelid::regclass AS tabela,
       conname AS restricao,
       CASE contype
         WHEN 'p' THEN 'PRIMARY KEY'
         WHEN 'u' THEN 'UNIQUE'
         WHEN 'c' THEN 'CHECK'
         WHEN 'f' THEN 'FOREIGN KEY' END AS tipo
FROM pg_constraint
WHERE connamespace = 'public'::regnamespace
  AND contype IN ('p', 'u', 'c', 'f')
ORDER BY conrelid::regclass::text, tipo, conname;

```

Fonte: Elaborado pelos autores (2026).

Quadro 10 – Restrições criadas no banco concessionaria_db

tabela	restricao	tipo
clientes	clientes_pkey	PRIMARY KEY
clientes	uq_cliente_cpf	UNIQUE

tabela	restricao	tipo
fornecedores	fornecedores_pkey	PRIMARY KEY
fornecedores	uq_fornecedor_cnpj	UNIQUE
funcionarios	ck_funcionario_salario	CHECK
funcionarios	funcionarios_pkey	PRIMARY KEY
veiculos	ck_veiculo_ano	CHECK
veiculos	ck_veiculo_preco	CHECK
veiculos	ck_veiculo_status	CHECK
veiculos	fk_veiculo_fornecedor	FOREIGN KEY
veiculos	veiculos_pkey	PRIMARY KEY
veiculos	uq_veiculo_placa	UNIQUE
vendas	ck_venda_valor	CHECK
vendas	fk_venda_cliente	FOREIGN KEY
vendas	fk_venda_funcionario	FOREIGN KEY
vendas	fk_venda_veiculo	FOREIGN KEY
vendas	vendas_pkey	PRIMARY KEY
vendas	uq_venda_veiculo	UNIQUE

Fonte: Elaborado pelos autores, a partir da execução no PostgreSQL 16.13 (2026).

5.3 CARGA DE DADOS

Com a estrutura pronta, a carga de dados foi feita com os três comandos da DML pedidos no trabalho: INSERT, para incluir linhas; UPDATE, para alterar linhas existentes; e DELETE, para removê-las. Todos os dados utilizados são fictícios.

5.3.1 Inserção de dados (INSERT)

O Código 8 insere os cadastros iniciais. Cada comando INSERT inclui várias linhas de uma vez: primeiro lista as colunas e, em seguida, os valores de cada linha, na mesma ordem. A coluna id não é informada, pois é gerada pelo próprio SGBD (coluna de identidade); por ser GENERATED ALWAYS, o PostgreSQL recusaria um valor informado manualmente. Na tabela veiculos, a coluna status também foi omitida e recebeu o valor padrão 'Disponível', e o Renault Kwid, por ser 0 km e ainda não emplacado, foi inserido com a placa NULL.

Código 8 – Inserção dos cadastros iniciais

```
INSERT INTO fornecedores (nome, cnpj, telefone) VALUES
  ('Auto Distribuidora Sul',      '12.345.678/0001-90', '(67) 3321-1122'),
  ('Veículos Brasil Ltda',        '98.765.432/0001-10', '(11) 4002-8922'),
```

```

('Centro-Oeste Automotores S.A.', '45.678.912/0001-33', '(67) 3025-4455');

-- A coluna status não é informada: recebe o valor padrão 'Disponível'.
-- O Kwid é 0 km e ainda não foi emplacado, por isso a placa fica NULL.
INSERT INTO veiculos
(marca, modelo, ano, cor, placa, preco, fornecedor_id)
VALUES
('Volkswagen', 'Gol', 2023, 'Branco', 'ABC1D23', 78900.00, 1),
('Chevrolet', 'Onix', 2024, 'Prata', 'DEF4E56', 85500.00, 1),
('Fiat', 'Toro', 2023, 'Vermelho', 'GHI7F89', 129900.00, 2),
('Toyota', 'Corolla', 2024, 'Preto', 'JKL0G12', 145000.00, 2),
('Hyundai', 'HB20', 2025, 'Cinza', 'MNO3H45', 92900.00, 3),
('Jeep', 'Compass', 2025, 'Branco', 'PQR6I78', 189900.00, 3),
('Renault', 'Kwid', 2026, 'Laranja', NULL, 74990.00, 1);

INSERT INTO clientes (nome, cpf, telefone, email, endereco) VALUES
('Mariana Souza', '123.456.789-00', '(67) 99911-2233',
'mariana.souza@email.com', 'Rua das Flores, 120 - Campo Grande/MS'),
('Ricardo Alves', '987.654.321-00', '(67) 98822-3344',
'ricardo.alves@email.com', 'Av. Afonso Pena, 500 - Campo Grande/MS'),
('Fernanda Oliveira', '456.789.123-00', '(67) 99155-4466',
'fernanda.oliveira@email.com', 'Rua 14 de Julho, 1800 - Campo Grande/MS'),
('Paulo Henrique Costa', '741.852.963-00', '(67) 98431-7788',
'paulo.costa@email.com', 'Rua Hayel Bon Faker, 950 - Dourados/MS');

INSERT INTO funcionarios (nome, cargo, salario, telefone) VALUES
('Bruno Lima', 'Vendedor', 2500.00, '(67) 99777-1111'),
('Camila Fernandes', 'Gerente de Vendas', 5200.00, '(67) 99666-2222'),
('Diego Rocha', 'Vendedor', 2500.00, '(67) 99555-3333');

```

Fonte: Elaborado pelos autores (2026).

Após as inserções, o banco passou a ter 3 fornecedores, 7 veículos, 4 clientes e 3 funcionários. O Quadro 11 mostra o conteúdo da tabela `veiculos` nesse momento, com os códigos gerados pela coluna de identidade, a situação preenchida pelo valor padrão e a placa nula do veículo 0 km.

Quadro 11 – Tabela `veiculos` após a inserção

id	marca	modelo	ano	cor	placa	preco	status	fornecedor_id
1	Volkswagen	Gol	2023	Branco	ABC1D23	78900.00	Disponível	1
2	Chevrolet	Onix	2024	Prata	DEF4E56	85500.00	Disponível	1
3	Fiat	Toro	2023	Vermelho	GHI7F89	129900.00	Disponível	2
4	Toyota	Corolla	2024	Preto	JKL0G12	145000.00	Disponível	2
5	Hyundai	HB20	2025	Cinza	MNO3H45	92900.00	Disponível	3
6	Jeep	Compass	2025	Branco	PQR6I78	189900.00	Disponível	3
7	Renault	Kwid	2026	Laranja	NULL	74990.00	Disponível	1

Fonte: Elaborado pelos autores, a partir da execução no PostgreSQL 16.13 (2026).

5.3.2 Registro das vendas (INSERT e UPDATE em transação)

Registrar uma venda exige duas operações: incluir a linha na tabela vendas e mudar a situação do veículo para 'Vendido'. Se apenas a primeira fosse gravada, por uma falha no meio do caminho, o veículo continuaria aparecendo como disponível mesmo já tendo sido vendido. Por isso, as duas operações foram agrupadas em uma transação, delimitada por BEGIN e COMMIT. Pela propriedade de atomicidade, ou todas as operações de uma transação são efetivadas no banco, ou nenhuma é (SILBERSCHATZ; KORTH; SUDARSHAN, 2012). O Código 9 registra as três vendas.

Código 9 – Registro das vendas em transações

```
-- Cada venda grava a linha em vendas e marca o veículo como vendido.
-- As duas operações ficam na mesma transação: ou as duas são
-- gravadas, ou nenhuma é.

BEGIN;
INSERT INTO vendas
    (cliente_id, funcionario_id, veiculo_id, data_venda, valor_venda)
VALUES (1, 1, 1, '2026-09-15', 77500.00);
UPDATE veiculos SET status = 'Vendido' WHERE id = 1;
COMMIT;

BEGIN;
INSERT INTO vendas
    (cliente_id, funcionario_id, veiculo_id, data_venda, valor_venda)
VALUES (3, 3, 4, '2026-09-17', 142000.00);
UPDATE veiculos SET status = 'Vendido' WHERE id = 4;
COMMIT;

BEGIN;
INSERT INTO vendas
    (cliente_id, funcionario_id, veiculo_id, data_venda, valor_venda)
VALUES (4, 1, 5, '2026-09-19', 91500.00);
UPDATE veiculos SET status = 'Vendido' WHERE id = 5;
COMMIT;
```

Fonte: Elaborado pelos autores (2026).

O Quadro 12 mostra a tabela vendas após o registro. Os veículos 1, 4 e 5 passaram à situação 'Vendido'.

Quadro 12 – Tabela vendas após o registro das vendas

id	cliente_id	funcionario_id	veiculo_id	data_venda	valor_venda
1	1	1	1	2026-09-15	77500.00
2	3	3	4	2026-09-17	142000.00
3	4	1	5	2026-09-19	91500.00

Fonte: Elaborado pelos autores, a partir da execução no PostgreSQL 16.13 (2026).

5.3.3 Atualização de dados (UPDATE)

O comando UPDATE altera colunas de linhas já existentes. A cláusula WHERE define quais linhas são afetadas; sem ela, todas as linhas da tabela seriam alteradas. O Código 10 traz quatro atualizações: o reajuste de preço de um veículo, a reserva de outro, a troca de telefone de uma cliente e um reajuste de 5% no salário de todos os vendedores, este último calculado a partir do valor atual da própria coluna.

Código 10 – Atualizações de dados

```
-- Reajuste de preço de um veículo específico
UPDATE veiculos
SET preco = 89900.00
WHERE modelo = 'Onix';

-- Reserva de um veículo para um cliente
UPDATE veiculos
SET status = 'Reservado'
WHERE placa = 'PQR6I78';

-- Atualização do telefone de um cliente
UPDATE clientes
SET telefone = '(67) 99911-9999'
WHERE cpf = '123.456.789-00';

-- Reajuste salarial de 5% para todos os vendedores
UPDATE funcionarios
SET salario = salario * 1.05
WHERE cargo = 'Vendedor';
```

Fonte: Elaborado pelos autores (2026).

O Quadro 13 compara os valores antes e depois das atualizações. O último comando afetou duas linhas, uma para cada vendedor.

Quadro 13 – Valores antes e depois das atualizações

Atualização	Linhas afetadas	Antes	Depois
Preço do Chevrolet Onix	1	preco = 85500.00	preco = 89900.00
Situação do Jeep Compass	1	status = Disponível	status = Reservado
Telefone de Mariana Souza	1	(67) 99911-2233	(67) 99911-9999
Salário dos vendedores (Bruno Lima e Diego Rocha)	2	salario = 2500.00	salario = 2625.00

Fonte: Elaborado pelos autores, a partir da execução no PostgreSQL 16.13 (2026).

5.3.4 Exclusão de dados (DELETE)

O comando DELETE remove linhas de uma tabela, também filtradas pela cláusula WHERE. O Código 11 exclui o Chevrolet Onix, devolvido ao fornecedor antes de ser vendido, e o cliente Ricardo Alves, que nunca realizou compras. As duas exclusões foram aceitas porque nenhuma venda faz referência a essas linhas.

Código 11 – Exclusões de dados

```
-- Remove um veículo que não foi vendido (devolvido ao fornecedor)
DELETE FROM veiculos
WHERE placa = 'DEF4E56';

-- Remove um cliente que nunca realizou compras
DELETE FROM clientes
WHERE cpf = '987.654.321-00';
```

Fonte: Elaborado pelos autores (2026).

5.4 TESTES DE INTEGRIDADE

Para confirmar que as restrições criadas protegem os dados, foram executados três comandos que violam regras de negócio (Código 12). No script, esses comandos ficam comentados, para não interromper a execução.

Código 12 – Comandos que violam as regras de negócio

```
-- a) Excluir um cliente que já comprou (bloqueado pela chave estrangeira)
DELETE FROM clientes WHERE cpf = '123.456.789-00';

-- b) Vender de novo um veículo já vendido (bloqueado pelo UNIQUE)
INSERT INTO vendas
(cliente_id, funcionario_id, veiculo_id, data_venda, valor_venda)
VALUES (3, 3, 1, '2026-09-21', 70000.00);

-- c) Gravar uma situação inexistente (bloqueado pelo CHECK)
UPDATE veiculos SET status = 'Vendida' WHERE id = 6;
```

Fonte: Elaborado pelos autores (2026).

Os três comandos foram recusados pelo SGBD, que devolveu as mensagens de erro reproduzidas no Quadro 14. Além da mensagem (ERROR), o PostgreSQL informa um detalhe (DETAIL) com o valor que causou o problema. As mensagens aparecem em inglês porque o servidor de testes estava configurado nesse idioma; em uma instalação em português, o texto vem traduzido.

Quadro 14 – Resultado dos testes de integridade

Teste	Regra	Resposta do PostgreSQL
a) Excluir a cliente Mariana Souza, que tem uma venda	RN07	ERROR: update or delete on table "clientes" violates foreign key constraint "fk_venda_cliente" on table "vendas" DETAIL: Key (id)=(1) is still referenced from table "vendas".
b) Vender novamente o Volkswagen Gol	RN06	ERROR: duplicate key value violates unique constraint "uq_venda_veiculo" DETAIL: Key (veiculo_id)=(1) already exists.
c) Gravar a situação 'Vendida', que não existe	RN04	ERROR: new row for relation "veiculos" violates check constraint "ck_veiculo_status" DETAIL: Failing row contains (6, Jeep, Compass, 2025, Branco, PQR6I78, 189900.00, Vendida, 3).

Fonte: Elaborado pelos autores, a partir da execução no PostgreSQL 16.13 (2026).

No teste a, a chave estrangeira `fk_venda_cliente`, com a ação `RESTRICT`, impediu a exclusão de um cliente que consta em uma venda. No teste b, a restrição `uq_venda_veiculo` impediu que o mesmo veículo fosse vendido duas vezes. No teste c, a restrição `ck_veiculo_status` recusou um valor fora da lista permitida. Nos três casos o banco permaneceu inalterado, o que demonstra que a integridade dos dados é garantida pelo próprio SGBD, independentemente da aplicação que venha a utilizá-lo.

5.5 CONSULTAS DE VERIFICAÇÃO

Por fim, o comando `SELECT` foi usado para conferir o resultado da carga. As consultas do Código 13 contam os registros de cada tabela, listam os veículos em estoque com o nome do fornecedor, reúnem os dados de cada venda a partir de quatro tabelas, por meio de junções (`JOIN`) pelas chaves estrangeiras, e somam o valor vendido por funcionário.

Código 13 – Consultas de verificação

```
-- Quantidade de registros em cada tabela
SELECT 'fornecedores' AS tabela, COUNT(*) AS registros FROM fornecedores
UNION ALL SELECT 'veiculos', COUNT(*) FROM veiculos
UNION ALL SELECT 'clientes', COUNT(*) FROM clientes
UNION ALL SELECT 'funcionarios', COUNT(*) FROM funcionarios
UNION ALL SELECT 'vendas', COUNT(*) FROM vendas;

-- Veículos em estoque (não vendidos), com o nome do fornecedor
SELECT v.marca, v.modelo, v.ano, v.preco, v.status,
       f.nome AS fornecedor
FROM veiculos v
LEFT JOIN fornecedores f ON f.id = v.fornecedor_id
```

```

WHERE v.status <> 'Vendido'
ORDER BY v.preco;

-- Vendas com o nome do cliente, do funcionário e o veículo vendido
SELECT vd.id, c.nome AS cliente, fu.nome AS funcionario,
       ve.marca || ' ' || ve.modelo AS veiculo,
       vd.data_venda, vd.valor_venda
FROM vendas vd
JOIN clientes c      ON c.id = vd.cliente_id
JOIN funcionarios fu  ON fu.id = vd.funcionario_id
JOIN veiculos ve     ON ve.id = vd.veiculo_id
ORDER BY vd.data_venda;

-- Total vendido por funcionário
SELECT fu.nome, fu.cargo,
       COUNT(vd.id) AS vendas,
       COALESCE(SUM(vd.valor_venda), 0.00) AS total_vendido
FROM funcionarios fu
LEFT JOIN vendas vd ON vd.funcionario_id = fu.id
GROUP BY fu.id, fu.nome, fu.cargo
ORDER BY total_vendido DESC;

```

Fonte: Elaborado pelos autores (2026).

Os resultados estão nos Quadros 15 a 18. Ao final da carga, o banco ficou com 3 fornecedores, 6 veículos, 3 clientes, 3 funcionários e 3 vendas (Quadro 15).

Quadro 15 – Quantidade de registros por tabela ao final da carga

tabela	registros
fornecedores	3
veiculos	6
clientes	3
funcionarios	3
vendas	3

Fonte: Elaborado pelos autores, a partir da execução no PostgreSQL 16.13 (2026).

A segunda consulta usa LEFT JOIN, e não JOIN, para que um veículo sem fornecedor (situação prevista na regra RN02) também apareça na lista de estoque (Quadro 16).

Quadro 16 – Veículos em estoque, com o fornecedor

marca	modelo	ano	preco	status	fornecedor
Renault	Kwid	2026	74990.00	Disponivel	Auto Distribuidora Sul
Fiat	Toro	2023	129900.00	Disponivel	Veículos Brasil Ltda
Jeep	Compass	2025	189900.00	Reservado	Centro-Oeste Automotores S.A.

Fonte: Elaborado pelos autores, a partir da execução no PostgreSQL 16.13 (2026).

A terceira consulta junta quatro tabelas para apresentar cada venda com os nomes do cliente e do funcionário e a identificação do veículo (Quadro 17).

Quadro 17 – Vendas realizadas

id	cliente	funcionario	veiculo	data_venda	valor_venda
1	Mariana Souza	Bruno Lima	Volkswagen Gol	2026-09-15	77500.00
2	Fernanda Oliveira	Diego Rocha	Toyota Corolla	2026-09-17	142000.00
3	Paulo Henrique Costa	Bruno Lima	Hyundai HB20	2026-09-19	91500.00

Fonte: Elaborado pelos autores, a partir da execução no PostgreSQL 16.13 (2026).

A última consulta agrupa as vendas por funcionário (GROUP BY) e usa as funções COUNT e SUM. Com LEFT JOIN, a gerente, que não registrou vendas, também aparece, com total zero (Quadro 18).

Quadro 18 – Total vendido por funcionário

nome	cargo	vendas	total_vendido
Bruno Lima	Vendedor	2	169000.00
Diego Rocha	Vendedor	1	142000.00
Camila Fernandes	Gerente de Vendas	0	0.00

Fonte: Elaborado pelos autores, a partir da execução no PostgreSQL 16.13 (2026).

5.6 RESUMO DOS COMANDOS UTILIZADOS

O Quadro 19 resume os comandos utilizados em cada etapa da implementação.

Quadro 19 – Comandos SQL utilizados na implementação

Etapa	Comandos	Subconjunto
Criação do banco	DROP DATABASE IF EXISTS, CREATE DATABASE ... WITH ENCODING	DDL
Estrutura de tabelas	CREATE TABLE com PRIMARY KEY, GENERATED ALWAYS AS IDENTITY, NOT NULL, UNIQUE, DEFAULT, CHECK e FOREIGN KEY	DDL
Inserção de dados	INSERT INTO ... VALUES	DML
Registro das vendas	BEGIN, INSERT, UPDATE, COMMIT	TCL e DML
Atualização de dados	UPDATE ... SET ... WHERE	DML
Exclusão de dados	DELETE FROM ... WHERE	DML
Verificação	SELECT com JOIN, LEFT JOIN, GROUP BY, COUNT, SUM, COALESCE e consulta ao catálogo pg_constraint	DQL

Fonte: Elaborado pelos autores (2026).

6 CONCLUSÃO

O trabalho cumpriu as quatro etapas propostas: o SGBD PostgreSQL 18 foi definido e justificado, o banco `concessionaria_db` foi criado, a estrutura de cinco tabelas foi implementada com chaves e restrições de integridade e a carga de dados foi realizada com os comandos `INSERT`, `UPDATE` e `DELETE`, todos documentados e executados com sucesso.

A modelagem em dois níveis mostrou-se importante para a implementação. O MER permitiu discutir as regras do negócio sem se preocupar com detalhes técnicos, e o DER, junto com o dicionário de dados, serviu de roteiro direto para os comandos `CREATE TABLE`. Decisões tomadas na modelagem, como a restrição `UNIQUE` na chave estrangeira `veiculo_id` e a ação `RESTRICT` nas exclusões, foram confirmadas nos testes de integridade: o próprio SGBD recusou a venda repetida de um veículo e a exclusão de um cliente com histórico de compras.

Também ficou evidente o valor das transações: ao agrupar o registro da venda e a atualização da situação do veículo, o banco nunca fica em um estado intermediário, em que um veículo vendido aparece como disponível.

Como trabalhos futuros, o modelo pode ser ampliado com tabelas de formas de pagamento e financiamento, agendamento de *test drive*, histórico de manutenção e comissões dos vendedores, além de gatilhos (*triggers*) que atualizem automaticamente a situação do veículo e visões (*views*) para relatórios gerenciais. O banco também está pronto para ser utilizado por uma aplicação com interface gráfica, que poderá ser desenvolvida em uma etapa posterior.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6023**: informação e documentação: referências: elaboração. Rio de Janeiro: ABNT, 2018.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 14724**: informação e documentação: trabalhos acadêmicos: apresentação. Rio de Janeiro: ABNT, 2011.

CHEN, Peter Pin-Shan. The entity-relationship model: toward a unified view of data. **ACM Transactions on Database Systems**, New York, v. 1, n. 1, p. 9-36, mar. 1976.

DATE, C. J. **Introdução a sistemas de bancos de dados**. Rio de Janeiro: Elsevier, 2004.

ELMASRI, Ramez; NAVATHE, Shamkant B. **Sistemas de banco de dados**. 7. ed. São Paulo: Pearson Education do Brasil, 2018.

HEUSER, Carlos Alberto. **Projeto de banco de dados**. 6. ed. Porto Alegre: Bookman, 2009.

SILBERSCHATZ, Abraham; KORTH, Henry F.; SUDARSHAN, S. **Sistema de banco de dados**. 6. ed. Rio de Janeiro: Elsevier, 2012.

THE POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 18 documentation**. [S. l.]: The PostgreSQL Global Development Group, 2026. Disponível em: <https://www.postgresql.org/docs/18/>. Acesso em: 22 set. 2026.

APÊNDICE A – SCRIPT DE CRIAÇÃO DO BANCO

Arquivo 01_criar_banco.sql, entregue junto com este documento. Deve ser executado conectado ao banco padrão postgres.

```
-- =====
-- BANCO DE DADOS DE UMA CONCESSIONÁRIA DE VEÍCULOS
-- SGBD: PostgreSQL 18 (compatível a partir da versão 10)
--
-- ARQUIVO 1 DE 2: CRIAÇÃO DO BANCO DE DADOS
-- Execute conectado ao banco padrão "postgres":
--   Terminal: psql -U postgres -f 01_criar_banco.sql
--   pgAdmin:  Query Tool no banco postgres; selecione UM comando
--             por vez e tecle F5 (o PostgreSQL não aceita DROP e
--             CREATE DATABASE enviados juntos no mesmo bloco)
-- =====

-- Apaga uma versão anterior para que o script possa ser executado
-- novamente sem erro
DROP DATABASE IF EXISTS concessionaria_db;

CREATE DATABASE concessionaria_db
  WITH ENCODING = 'UTF8'
       TEMPLATE = template0;
```

APÊNDICE B – SCRIPT DE ESTRUTURA E CARGA DE DADOS

Arquivo 02_tabelas_e_carga.sql, entregue junto com este documento. Deve ser executado conectado ao banco concessionaria_db, criado pelo script do Apêndice A. Executado de uma só vez, ele cria as tabelas e realiza toda a carga de dados descrita na seção 5.

```
-- =====
-- BANCO DE DADOS DE UMA CONCESSIONÁRIA DE VEÍCULOS
-- SGBD: PostgreSQL 18 (compatível a partir da versão 10)
--
-- ARQUIVO 2 DE 2: ESTRUTURA DE TABELAS E CARGA DE DADOS
-- Execute conectado ao banco concessionaria_db:
--   pgAdmin: Query Tool no banco concessionaria_db, F5
--   Terminal: psql -U postgres -d concessionaria_db -f 02_tabelas_e_carga.sql
-- =====

-- -----
-- 2) CRIAÇÃO DA ESTRUTURA DE TABELAS (DDL)
--   As tabelas referenciadas são criadas antes das que as
--   referenciam por chave estrangeira.
-- -----

CREATE TABLE fornecedores (
    id          INT          GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nome        VARCHAR(100) NOT NULL,
    cnpj        VARCHAR(18)  NOT NULL,
    telefone    VARCHAR(15),
    CONSTRAINT uq_fornecedor_cnpj UNIQUE (cnpj)
);

CREATE TABLE veiculos (
    id          INT          GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    marca       VARCHAR(50)  NOT NULL,
    modelo      VARCHAR(50)  NOT NULL,
    ano         SMALLINT     NOT NULL,
    cor         VARCHAR(30),
    placa       VARCHAR(8),
    preco       NUMERIC(10,2) NOT NULL,
    status      VARCHAR(20)  NOT NULL DEFAULT 'Disponivel',
    fornecedor_id INT,
    CONSTRAINT uq_veiculo_placa UNIQUE (placa),
    CONSTRAINT ck_veiculo_ano CHECK (ano BETWEEN 1900 AND 2100),
    CONSTRAINT ck_veiculo_preco CHECK (preco > 0),
    CONSTRAINT ck_veiculo_status
        CHECK (status IN ('Disponivel', 'Reservado', 'Vendido')),
    CONSTRAINT fk_veiculo_fornecedor
        FOREIGN KEY (fornecedor_id) REFERENCES fornecedores (id)
        ON DELETE SET NULL
        ON UPDATE CASCADE
);
```

```

CREATE TABLE clientes (
    id          INT          GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nome        VARCHAR(100) NOT NULL,
    cpf         VARCHAR(14)  NOT NULL,
    telefone    VARCHAR(15),
    email       VARCHAR(100),
    endereco    VARCHAR(150),
    CONSTRAINT uq_cliente_cpf UNIQUE (cpf)
);

CREATE TABLE funcionarios (
    id          INT          GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nome        VARCHAR(100) NOT NULL,
    cargo       VARCHAR(50)  NOT NULL,
    salario     NUMERIC(10,2) NOT NULL,
    telefone    VARCHAR(15),
    CONSTRAINT ck_funcionario_salario CHECK (salario > 0)
);

CREATE TABLE vendas (
    id          INT          GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    cliente_id  INT          NOT NULL,
    funcionario_id INT      NOT NULL,
    veiculo_id  INT          NOT NULL,
    data_venda  DATE         NOT NULL,
    valor_venda NUMERIC(10,2) NOT NULL,
    CONSTRAINT uq_venda_veiculo UNIQUE (veiculo_id),
    CONSTRAINT ck_venda_valor CHECK (valor_venda > 0),
    CONSTRAINT fk_venda_cliente
        FOREIGN KEY (cliente_id) REFERENCES clientes (id)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_venda_funcionario
        FOREIGN KEY (funcionario_id) REFERENCES funcionarios (id)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_venda_veiculo
        FOREIGN KEY (veiculo_id) REFERENCES veiculos (id)
        ON DELETE RESTRICT ON UPDATE CASCADE
);

-- Confere as restrições criadas, consultando o catálogo do PostgreSQL
-- (o filtro por contype deixa de fora as NOT NULL, que o PostgreSQL 18
-- também registra no pg_constraint)
SELECT conrelid::regclass AS tabela,
       conname AS restricao,
       CASE contype WHEN 'p' THEN 'PRIMARY KEY'
                   WHEN 'u' THEN 'UNIQUE'
                   WHEN 'c' THEN 'CHECK'
                   WHEN 'f' THEN 'FOREIGN KEY' END AS tipo
FROM pg_constraint
WHERE connamespace = 'public'::regnamespace
AND contype IN ('p', 'u', 'c', 'f')
ORDER BY conrelid::regclass::text, tipo, conname;

```

```

-----
-- 3) CARGA DE DADOS (DML)
-----

```

```
-- 3.1) INSERT: cadastros iniciais (dados fictícios)

INSERT INTO fornecedores (nome, cnpj, telefone) VALUES
  ('Auto Distribuidora Sul',      '12.345.678/0001-90', '(67) 3321-1122'),
  ('Veículos Brasil Ltda',       '98.765.432/0001-10', '(11) 4002-8922'),
  ('Centro-Oeste Automotores S.A.', '45.678.912/0001-33', '(67) 3025-4455');

-- A coluna status não é informada: recebe o valor padrão 'Disponível'.
-- O Kwid é 0 km e ainda não foi emplacado, por isso a placa fica NULL.
INSERT INTO veiculos
  (marca, modelo, ano, cor, placa, preco, fornecedor_id)
VALUES
  ('Volkswagen', 'Gol',      2023, 'Branco',   'ABC1D23', 78900.00, 1),
  ('Chevrolet',  'Onix',     2024, 'Prata',   'DEF4E56', 85500.00, 1),
  ('Fiat',       'Toro',     2023, 'Vermelho', 'GHI7F89', 129900.00, 2),
  ('Toyota',     'Corolla',  2024, 'Preto',   'JKL0G12', 145000.00, 2),
  ('Hyundai',    'HB20',     2025, 'Cinza',   'MNO3H45', 92900.00, 3),
  ('Jeep',       'Compass',  2025, 'Branco',   'PQR6I78', 189900.00, 3),
  ('Renault',    'Kwid',     2026, 'Laranja',  NULL,      74990.00, 1);

INSERT INTO clientes (nome, cpf, telefone, email, endereco) VALUES
  ('Mariana Souza', '123.456.789-00', '(67) 99911-2233',
   'mariana.souza@email.com', 'Rua das Flores, 120 - Campo Grande/MS'),
  ('Ricardo Alves', '987.654.321-00', '(67) 98822-3344',
   'ricardo.alves@email.com', 'Av. Afonso Pena, 500 - Campo Grande/MS'),
  ('Fernanda Oliveira', '456.789.123-00', '(67) 99155-4466',
   'fernanda.oliveira@email.com', 'Rua 14 de Julho, 1800 - Campo Grande/MS'),
  ('Paulo Henrique Costa', '741.852.963-00', '(67) 98431-7788',
   'paulo.costa@email.com', 'Rua Hayel Bon Faker, 950 - Dourados/MS');

INSERT INTO funcionarios (nome, cargo, salario, telefone) VALUES
  ('Bruno Lima',      'Vendedor',      2500.00, '(67) 99777-1111'),
  ('Camila Fernandes', 'Gerente de Vendas', 5200.00, '(67) 99666-2222'),
  ('Diego Rocha',     'Vendedor',      2500.00, '(67) 99555-3333');

-- 3.2) INSERT + UPDATE: registro das vendas
-- Cada venda grava a linha em vendas e marca o veículo como vendido.
-- As duas operações ficam na mesma transação: ou as duas são
-- gravadas, ou nenhuma é.

BEGIN;
INSERT INTO vendas
  (cliente_id, funcionario_id, veiculo_id, data_venda, valor_venda)
VALUES (1, 1, 1, '2026-09-15', 77500.00);
UPDATE veiculos SET status = 'Vendido' WHERE id = 1;
COMMIT;

BEGIN;
INSERT INTO vendas
  (cliente_id, funcionario_id, veiculo_id, data_venda, valor_venda)
VALUES (3, 3, 4, '2026-09-17', 142000.00);
UPDATE veiculos SET status = 'Vendido' WHERE id = 4;
COMMIT;

BEGIN;
```

```

INSERT INTO vendas
    (cliente_id, funcionario_id, veiculo_id, data_venda, valor_venda)
VALUES (4, 1, 5, '2026-09-19', 91500.00);
UPDATE veiculos SET status = 'Vendido' WHERE id = 5;
COMMIT;

-- 3.3) UPDATE: atualização de dados

-- Reajuste de preço de um veículo específico
UPDATE veiculos
SET preco = 89900.00
WHERE modelo = 'Onix';

-- Reserva de um veículo para um cliente
UPDATE veiculos
SET status = 'Reservado'
WHERE placa = 'PQR6I78';

-- Atualização do telefone de um cliente
UPDATE clientes
SET telefone = '(67) 99911-9999'
WHERE cpf = '123.456.789-00';

-- Reajuste salarial de 5% para todos os vendedores
UPDATE funcionarios
SET salario = salario * 1.05
WHERE cargo = 'Vendedor';

-- 3.4) DELETE: remoção de dados

-- Remove um veículo que não foi vendido (devolvido ao fornecedor)
DELETE FROM veiculos
WHERE placa = 'DEF4E56';

-- Remove um cliente que nunca realizou compras
DELETE FROM clientes
WHERE cpf = '987.654.321-00';

-----
-- 4) TESTES DE INTEGRIDADE
-- Os comandos abaixo DEVEM falhar: é o SGBD protegendo os dados.
-- Estão comentados para não interromper o script; para testar,
-- retire o "-- " do início das linhas e execute só aquele comando.
-----

-- a) Excluir um cliente que já comprou (bloqueado pela chave estrangeira)
-- DELETE FROM clientes WHERE cpf = '123.456.789-00';

-- b) Vender de novo um veículo já vendido (bloqueado pelo UNIQUE)
-- INSERT INTO vendas
--     (cliente_id, funcionario_id, veiculo_id, data_venda, valor_venda)
-- VALUES (3, 3, 1, '2026-09-21', 70000.00);

-- c) Gravar uma situação inexistente (bloqueado pelo CHECK)
-- UPDATE veiculos SET status = 'Vendida' WHERE id = 6;

```

```

-----
-- 5) CONSULTAS DE VERIFICAÇÃO
-----

-- Quantidade de registros em cada tabela
SELECT 'fornecedores' AS tabela, COUNT(*) AS registros FROM fornecedores
UNION ALL SELECT 'veiculos', COUNT(*) FROM veiculos
UNION ALL SELECT 'clientes', COUNT(*) FROM clientes
UNION ALL SELECT 'funcionarios', COUNT(*) FROM funcionarios
UNION ALL SELECT 'vendas', COUNT(*) FROM vendas;

-- Veículos em estoque (não vendidos), com o nome do fornecedor
SELECT v.marca, v.modelo, v.ano, v.preco, v.status,
       f.nome AS fornecedor
FROM veiculos v
LEFT JOIN fornecedores f ON f.id = v.fornecedor_id
WHERE v.status <> 'Vendido'
ORDER BY v.preco;

-- Vendas com o nome do cliente, do funcionário e o veículo vendido
SELECT vd.id, c.nome AS cliente, fu.nome AS funcionario,
       ve.marca || ' ' || ve.modelo AS veiculo,
       vd.data_venda, vd.valor_venda
FROM vendas vd
JOIN clientes c ON c.id = vd.cliente_id
JOIN funcionarios fu ON fu.id = vd.funcionario_id
JOIN veiculos ve ON ve.id = vd.veiculo_id
ORDER BY vd.data_venda;

-- Total vendido por funcionário
SELECT fu.nome, fu.cargo,
       COUNT(vd.id) AS vendas,
       COALESCE(SUM(vd.valor_venda), 0.00) AS total_vendido
FROM funcionarios fu
LEFT JOIN vendas vd ON vd.funcionario_id = fu.id
GROUP BY fu.id, fu.nome, fu.cargo
ORDER BY total_vendido DESC;

```